

Lab 1: Next.js Workspace Init & The App Router File System

Step 0 : Create a nx workspace

```
pnpm create nx-workspace@latest next-lab-workspace --preset=apps  
  
cd next-lab-workspace  
  
pnpm approve-builds  
  
pnpm add -D -w @nx/next  
  
pnpm approve-builds  
  
pnpm nx g @nx/next:app apps/frontend --appDir=true --srcDir=false --  
e2eTestRunner=none  
  
pnpm nx show projects  
  
pnpm nx dev frontend
```

Add the following line to the `tsconfig.base.json` file in "compilerOptions"

```
"jsx": "react-jsx",
```

Step 1.1: Verify Your New Next.js App Router Structure

Since we successfully scaffolded the Next.js app in the previous step, verify that your file tree matches the official Next.js App Router convention:

Check the directory layout under `apps/frontend`

```
ls -R apps/frontend/src/app
```

You should see `layout.tsx` and `page.tsx` files directly inside the root app folder.

Step 1.2: Create a Nested Route with Shared Layouts

Let's build a nested Dashboard view (`/dashboard`) that inherits a global layout but introduces its own sub-layout.

Create the nested folders:

```
mkdir -p apps/frontend/src/app/dashboard
```

Step 1.3: Create the Dashboard Sub-Layout

Create a unique layout that only wraps files inside the dashboard subsystem.

Create `apps/frontend/src/app/dashboard/layout.tsx` and add this code:

```
export default function DashboardLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <section style={{ border: '2px dashed #4F46E5', padding: '1rem',
marginTop: '1rem' }}>
      <h2 style={{ color: '#4F46E5' }}>Dashboard Sub-Layout</h2>
      {children}
    </section>
  );
}
```

Step 1.4: Create the Dashboard Core Page

Create the entry point file `apps/frontend/src/app/dashboard/page.tsx` and add this code:

```
export default function DashboardPage() {
  return (
    <div>
      <h3>Main Dashboard Content Area</h3>
      <p>This page is deeply nested within the application router
structure.</p>
    </div>
  );
}
```

Step 1.5: Verify Route Execution

Launch the local Next.js dev engine:

```
pnpm nx dev frontend
```

Open your browser and navigate to `http://localhost:4200/dashboard` to verify that both the global layout and the new dashboard sub-layout wrap your page contents together.

Lab 2: Shared Component Mapping with React Server Components (RSC)

Step 2.1: Generate a Shared UI Library

In a standard monorepo, UI layouts should be decoupled into a shared library.

Create a lightweight JavaScript/TypeScript workspace library to hold our static server components:

```
pnpm nx g @nx/js:lib libs/shared-ui --bundler=none --unitTestRunner=jest
```

Step 2.2: Add a Pure React Server Component (RSC)

Create a profile banner component inside this library. Because App Router components are Server Components by default, this will execute purely on the backend server during requests.

Create `libs/shared-ui/src/lib/server-banner.tsx`:

```
export function ServerBanner() {
  // This log will print in your terminal log, NEVER in the browser console
  console.log('>>> [RSC Execution]: Rendering ServerBanner on the server node.');
```

```
  return (
    <div style={{ background: '#F3F4F6', padding: '1.5rem', borderRadius:
'6px', margin: '1rem 0' }}>
      <h4>Secured React Server Component (RSC)</h4>
      <p>This banner component was fetched, interpreted, and compiled
entirely on the server. Zero JS was sent to the client browser for this
layout element.</p>
    </div>
  );
}
```

Step 2.3: Export the Component via the Library Index

Open `libs/shared-ui/src/index.ts` and add this export statement:

```
export * from './lib/server-banner';
```

Step 2.4: Integrate the RSC into Your Dashboard

Open your nested page file `apps/frontend/src/app/dashboard/page.tsx`. Import your new component using the automated alias generated by Nx:

```
import { ServerBanner } from '@next-lab-workspace/shared-ui';
export default function DashboardPage() {
  return (
    <div>
      <h3>Main Dashboard Content Area</h3>
      <p>This page is deeply nested within the application router
structure.</p>

      {/* Injecting the Server Component */}
      <ServerBanner />
    </div>
  );
}
```

Step 2.5: Test and Check Your Terminal Output

Run the Next.js app dev command again:

```
pnpm nx dev frontend
```

1. Open <http://localhost:4200/dashboard>.
2. Inspect your browser development console—you will notice the log message is completely missing.
3. Check your system terminal window—you will see the message `>>> [RSC Execution]...` printed, confirming the component executed natively on the server block.

Lab 3: Client-Side Interactivity & Hydration Boundaries

In this lab, you will learn how to introduce client-side React features (like hooks, state, and event listeners) without turning your entire application into a heavy client-side bundle.

Step 3.1: Create an Interactive Component in the Shared UI Library

We will create a counter component that requires user interaction. Because it uses state (`useState`) and click handlers, it cannot be rendered exclusively on the server.

Create a new file named `libs/shared-ui/src/lib/interactive-counter.tsx` and add this code:

```
'use client'; // This directive tells Next.js this is the hydration
boundary entry point
import { useState } from 'react';
export function InteractiveCounter() {
  const [count, setCount] = useState(0);

  // This log will print in BOTH the server terminal (pre-rendering) AND
  the browser console (interactivity)
  console.log(`>>> [Hydration Boundary]: Counter rendering with value:
```

```

    ${count}`);

    return (
      <div style={{ border: '1px solid #10B981', padding: '1.5rem',
borderRadius: '6px', margin: '1rem 0' }}>
        <h4 style={{ color: '#10B981' }}> Interactive Client Component</h4>
        <p>This box requires client-side JavaScript to track state
modifications.</p>
        <p>Current Hits: <strong>{count}</strong></p>
        <button
          onClick={() => setCount(count + 1)}
          style={{ background: '#10B981', color: 'white', border: 'none',
padding: '0.5rem 1rem', borderRadius: '4px', cursor: 'pointer' }}
        >
          Increment Counter
        </button>
      </div>
    );
  }
}

```

Step 3.2: Export the Interactive Component

Open your library entry point file, `libs/shared-ui/src/index.ts`, and append the new component export:

```

export * from './lib/server-banner';
export * from './lib/interactive-counter'; // Add this line

```

Step 3.3: Nest the Client Component inside your Server Dashboard

A key rule of the App Router is that you can import Client Components into Server Components, but you cannot import Server Components into Client Components without passing them as children.

Open `apps/frontend/src/app/dashboard/page.tsx` and import the interactive component alongside your server banner:

```

import { ServerBanner, InteractiveCounter } from '@next-lab-
workspace/shared-ui';
export default function DashboardPage() {
  return (
    <div>
      <h3>Main Dashboard Content Area</h3>
      <p>This page is deeply nested within the application router
structure.</p>

      {/* React Server Component (No client JS) */}
      <ServerBanner />

      {/* React Client Component (Hydrated on the client) */}

```

```
    <InteractiveCounter />
  </div>
);
}
```

Step 3.4: Observe Hydration and Execution Logs

Boot up your Next.js application dev server:

```
pnpm nx dev frontend
```

1. Open your browser and navigate to <http://localhost:4200/dashboard>.
2. Look at your System Terminal Window. You will see the counter log print once because Next.js pre-renders the client component layout to HTML on the server before sending it over.
3. Open your Browser Developer Console (F12). You will see the counter log print there as well, showing that hydration was successful.
4. Click the "Increment Counter" button. The count updates instantly, and a new log entry fires exclusively in the browser console.

Step 3.5: Verify the Monorepo Architecture Graph

Run the Nx graph tool to verify that both components are tracked accurately across your code layout boundaries:

```
pnpm nx graph
```

Your frontend node will maintain its arrow to shared-ui, keeping the core application architecture perfectly modular and clean.

Lab 4: Static Site Generation (SSG) with generateStaticParams

In this lab, you will learn how to configure Next.js to fetch data during the project compilation phase and pre-render dynamic, parameterised parameters into high-performance, static HTML files using `generateStaticParams()`.

Step 4.1: Create a Dynamic Route Segment Structure

We will build a dynamic inventory or product listing subsystem under `/products/[id]`.

First, create the required nested directory structure inside your application router folder:

```
mkdir -p "apps/frontend/src/app/products/[id]"
```

Step 4.2: Build the Static Data Fetcher and Page Component

Create a page file that mimics a database query using standard `fetch()` during compilation, then exports `generateStaticParams` to tell Next.js exactly which product IDs to hardcode into static assets.

Create a new file named `apps/frontend/src/app/products/[id]/page.tsx` and add this code:

```
// Mock database array containing mock data objects
const PRODUCT_DATABASE = [
  { id: '101', name: 'Premium Wireless Headphones', price: '₹14,999' },
  { id: '102', name: 'Ergonomic Mechanical Keyboard', price: '₹8,499' },
  { id: '103', name: 'Ultra-Wide Gaming Monitor', price: '₹32,999' },
];
// 1. Tell Next.js which dynamic parameter paths to bake at build time
export async function generateStaticParams() {
  console.log('>>> [SSG Engine]: Generating static parameters for product paths.');
```

```
  // Maps to an array of objects representing the [id] segment: [{ id:
  '101' }, ...]
  return PRODUCT_DATABASE.map((product) => ({
    id: product.id,
  }));
}
// 2. The React Server Component that renders individual product layouts
export default async function ProductPage({ params }: { params: Promise<{
id: string }> }) {
  const { id } = await params;
  const product = PRODUCT_DATABASE.find((p) => p.id === id);

  if (!product) {
    return <div style={{ padding: '2rem' }}><h3>Product Not Found</h3>
</div>;
  }

  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif' }}>
      <h2 style={{ color: '#2563EB' }}> Static Product Showcase (SSG)</h2>
      <p>This layout asset was fully pre-compiled into static HTML during
      compilation.</p>

      <div style={{ border: '1px solid #2563EB', padding: '1.5rem',
      borderRadius: '8px', background: '#FFF6FF', maxWidth: '400px' }}>
        <h3>{product.name}</h3>
        <p><strong>Product ID Reference:</strong> {product.id}</p>
        <p><strong>Market Value:</strong> {product.price}</p>
      </div>
    </div>
  );
}
```

Step 4.3: Execute the Production Compiling Phase

To see Static Site Generation in action, we need to compile the production bundle. Nx will run the inferred Next.js build target.

```
pnpm nx build frontend
```

Step 4.4: Inspect the Build Analytics Manifest

Look closely at the console output logs generated by Next.js at the end of the compilation sequence. You will see an analysis breakdown symbol table similar to this:

```
Route (app)          Size      First Load JS
├─ ○ /                171 B      87.3 kB
├─ ○ /dashboard       212 B      91.2 kB
└─ ● /products/[id]   142 B      87.3 kB
    └─ /products/101
      └─ /products/102
        └─ /products/103

○ (Static)    prerendered as static content
● (SSG)       prerendered as static HTML (uses generateStaticParams)
```

Notice the filled circle symbol (●) next to `/products/[id]`. This confirms Next.js successfully intercepted your data array and spit out three standalone, hyper-optimized HTML pages.

Step 4.5: Preview the Compiled Static Sites

Run the preview server to launch your compiled production assets locally:

```
pnpm nx start frontend
```

Open your browser to <http://localhost:4200/products/101> and <http://localhost:4200/products/102> to confirm the static pages render with instant execution response times.

Lab 5: Server-Side Rendering (SSR) with Dynamic Data Caching

In this lab, you will learn how to bypass static generation for pages that require real-time, volatile, or user-specific datasets. You will use Next.js's dynamic rendering rules by forcing cache exclusions (`cache: 'no-store'`) or bypassing static tracking using the `dynamic = 'force-dynamic'` property.

Step 5.1: Create a Dynamic Live Route Folder

We will build a dynamic crypto-ticker or live system dashboard under `/live-metrics`. This view cannot be pre-built because it must pull the exact timestamp and changing values on every single user request.

Create the target folder inside your application space:

```
mkdir -p apps/frontend/src/app/live-metrics
```

Step 5.2: Build the Server-Side Rendered (SSR) Component

Create a page that forces dynamic execution. By using the force-dynamic configuration directive, you inform the Next.js compiler that this route must skip static builds and run freshly on the Node server loop for every page hit.

Create a new file named `apps/frontend/src/app/live-metrics/page.tsx` and add this code:

```
// Force Next.js to switch this route from Static (o) to Live Server-Side
Rendered (f)
export const dynamic = 'force-dynamic';
async function getLiveMarketMetrics() {
  // Simulating an active api call to an external live tracking system
  const timestamp = new Date().toLocaleTimeString('en-IN', { timeZone:
'Asia/Kolkata' });
  const mockGasPrice = Math.floor(Math.random() * (120 - 20 + 1)) + 20;

  return {
    timestamp,
    mockGasPrice,
  };
}
export default async function LiveMetricsPage() {
  const metrics = await getLiveMarketMetrics();

  console.log(`>>> [SSR Compute Engine]: Generating fresh metrics page at
${metrics.timestamp}`);

  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif' }}>
      <h2 style={{ color: '#DC2626' }}> Live Compute Dashboard (SSR)</h2>
      <p>This layout block is reconstructed on the server node during every
single browser request.</p>

      <div style={{ border: '1px solid #DC2626', padding: '1.5rem',
borderRadius: '8px', background: '#FEF2F2', maxWidth: '400px' }}>
        <h3>Network Health Stats</h3>
        <p><strong>Server Render Time:</strong> {metrics.timestamp}</p>
        <p><strong>Estimated Gas Cost:</strong> {metrics.mockGasPrice}
Gwei</p>
        <p style={{ fontSize: '0.85rem', color: '#991B1B', fontStyle:
'italic' }}>
          *Refresh the page to force the server to recalculate the metrics
```

```
above.  
    </p>  
  </div>  
</div>  
)  
};  
}
```

Step 5.3: Run the Production Compiling Sweep

Let's re-compile the production bundle to observe how Next.js explicitly categorises this route differently compared to the static paths built in Lab 4.

```
pnpm nx build frontend
```

Step 5.4: Validate the Route Type Mapping Manifest

Look at the generated console symbol key table at the conclusion of the build step:

Route (app)	Size	First Load JS
○ /	171 B	87.3 kB
○ /dashboard	212 B	91.2 kB
f /live-metrics	314 B	87.3 kB
● /products/[id]	142 B	87.3 kB

○ (Static)	prerendered as static content
● (SSG)	prerendered as static HTML (uses generateStaticParams)
f (Dynamic)	server-rendered on demand (uses live runtime server)

Observe the function symbol (f) next to /live-metrics. This indicates that Next.js successfully mapped this path out to execute completely on demand.

Step 5.5: Test the Live SSR Component Behavior

Run the preview server layer to spin up your application runtime setup locally:

```
pnpm nx start frontend
```

1. Open your browser and navigate to <http://localhost:4200/live-metrics>.
 2. Observe the rendered time metrics and gas costs.
 3. Refresh the web page multiple times—the time values and gas metrics update instantly on every reload.
 4. Check your system command terminal logs—you will see the >>> [SSR Compute Engine]... statement log out freshly on every refresh, confirming live server-side execution.
-

Lab 6: Incremental Static Regeneration (ISR)

In this lab, you will learn how to build a high-performance hybrid page that is cached statically at build time but automatically re-validates and updates itself in the background in production when data changes—without forcing a full monorepo build or redeployment cycle.

Step 6.1: Create an ISR Route Directory

We will create a dynamic announcement or blog billboard under `/announcements`. This content changes occasionally but does not need live calculation for every user request.

Create the target folder inside your application space:

```
mkdir -p apps/frontend/src/app/announcements
```

Step 6.2: Build the ISR Component

To enable ISR, you configure a route segment configuration property named `revalidate`. By setting this value, Next.js serves the cached static page instantly but checks if background data needs to be re-fetched after a designated period.

Create a new file named `apps/frontend/src/app/announcements/page.tsx` and add this code:

```
// Force Next.js to revalidate this page's static cache at most every 10
seconds
export const revalidate = 10;
async function getLatestAnnouncement() {
  const fetchTime = new Date().toLocaleTimeString('en-IN', { timeZone:
'Asia/Kolkata' });

  // In a real application, you would pull this from a CMS or external
database API
  const headlines = [
    "Monorepo migration successful across all engineering tracks!",
    "Workspace architecture upgraded to the latest Next.js and Nx
frameworks.",
    "System maintenance complete: Cache throughput increased by 40%."
  ];

  // Randomly rotate headlines to simulate content modifications in the
database
  const randomHeadline = headlines[Math.floor(Math.random() *
headlines.length)];

  return {
    fetchTime,
    headline: randomHeadline,
  };
}
```

```

export default async function AnnouncementsPage() {
  const data = await getLatestAnnouncement();

  console.log(`>>> [ISR Engine]: Rendering/Revalidating page layout. Fetch
time: ${data.fetchTime}`);

  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif' }}>
      <h2 style={{ color: '#D97706' }}><img alt="Hybrid Static Regeneration (ISR) icon" data-bbox="515 185 535 200"/> Hybrid Static Regeneration (ISR)
    </h2>
    <p>This layout combines the speed of SSG with background runtime
content freshness.</p>

    <div style={{ border: '1px solid #D97706', padding: '1.5rem',
borderRadius: '8px', background: '#FEF3C7', maxWidth: '500px' }}>
      <h3>Broadcast Board</h3>
      <p><strong>Current Bulletin:</strong> "{data.headline}"</p>
      <hr style={{ border: '0', borderTop: '1px solid #F59E0B', margin:
'1rem 0' }} />
      <p style={{ fontSize: '0.85rem', color: '#92400E' }}>
        <strong>Cache Compilation Stamp:</strong> {data.fetchTime}
      </p>
      <p style={{ fontSize: '0.8rem', color: '#B45309', fontStyle:
'italic', marginTop: '0.5rem' }}>
        *Refreshing will serve the cached view instantly. If more than 10
seconds have passed, your click will trigger an asynchronous background
update for the NEXT visitor.
      </p>
    </div>
  </div>
);
}

```

Step 6.3: Run the Production Compiling Sweep

Compile the workspace production bundles to see how Next.js marks an ISR configuration.

```
pnpm nx build frontend
```

Step 6.4: Validate the Route Type Mapping Manifest

Look closely at your output manifest at the conclusion of the compilation process. Next.js marks ISR configurations with a clean timestamp designation:

Route (app)	Size	First Load JS
└ ○ /	171 B	87.3 kB
└ ○ /dashboard	212 B	91.2 kB
└ ○ /announcements (ISR: 10s)	280 B	87.3 kB

└─ f /live-metrics	314 B	87.3 kB
└─ ● /products/[id]	142 B	87.3 kB

Notice that `/announcements` is grouped alongside static routes but highlights a clear operational (ISR: 10s) rule pipeline.

Step 6.5: Verify Background Cache Updates

Run your local production preview server layer:

```
pnpm nx start frontend
```

1. Open your browser and navigate to `http://localhost:4200/announcements`.
2. Observe the current announcement text and the Cache Compilation Stamp.
3. Refresh the page immediately multiple times—the time and text remain completely frozen, showing that Next.js is serving it instantly out of the static cache.
4. Wait more than 10 seconds, then trigger a page refresh. The page will still display the old timestamp because Next.js hands you the cached page instantly to ensure zero lag, but it fires the data pipeline in the background. Check your terminal logs to see the `>>> [ISR Engine]... log activate`.
5. Refresh the page one more time—you will now see the updated timestamp and a fresh headline, proving the background cache update was successfully pushed to the user interface.

Lab 7: Streamed Rendering with Suspense Boundaries

In this lab, you will learn how to handle slow asynchronous data fetches (like a sluggish database query or a slow external API) without blocking the entire page layout from rendering. You will use Next.js's native streaming capabilities along with React Suspense boundaries and a specialized loading skeleton file (`loading.tsx`).

Step 7.1: Create a Slow Loading Route Folder

We will build a mocked dashboard page that loads various analytics. One of the data panels will intentionally run a delayed promise to simulate network latency.

Create the target folder inside your application space:

```
mkdir -p apps/frontend/src/app/analytics
```

Step 7.2: Build the Main Page with Blocked Sub-Components

Create a page that hosts two components: one that loads instantly, and another that requires an explicit data fetch delay.

Create a new file named `apps/frontend/src/app/analytics/page.tsx` and add this code:

```
import { Suspense } from 'react';
// 1. A fast component that requires no asynchronous wait time
function QuickSummaryPanel() {
  return (
    <div style={{ background: '#E0F2FE', padding: '1rem', borderRadius:
'6px', marginBottom: '1rem', border: '1px solid #0284C7' }}>
      <h4>Fast System Check</h4>
      <p>Workspace operations status: **Optimal** (Loaded instantly)</p>
    </div>
  );
}

// 2. A simulated slow data fetching function
async function getSlowDatabaseReport() {
  // Simulate a 3.5-second backend database delay
  await new Promise((resolve) => setTimeout(resolve, 3500));
  return {
    totalRevenue: '₹4,52,900',
    activeSubscribers: '12,450',
  };
}

// 3. The asynchronous server component that waits on data
async function HeavyAnalyticsPanel() {
  const data = await getSlowDatabaseReport();
  return (
    <div style={{ background: '#ECFDF5', padding: '1.5rem', borderRadius:
'6px', border: '1px solid #059669' }}>
      <h4>Deep Database Analytics Report</h4>
      <p><strong>Total Monorepo Revenue:</strong> {data.totalRevenue}</p>
      <p><strong>Active Subscribers:</strong> {data.activeSubscribers}</p>
    </div>
  );
}

// 4. A lightweight fallback skeleton loader
function SkeletonLoader() {
  return (
    <div style={{ background: '#F3F4F6', padding: '1.5rem', borderRadius:
'6px', border: '1px dashed #9CA3AF', animation: 'pulse 1.5s infinite' }}>
      <p style={{ color: '#6B7280', fontWeight: 'bold' }}> Querying
database... Streaming data packets... Please wait...</p>
    </div>
  );
}

// 5. Parent Server Layout Component
export default function AnalyticsPage() {
  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif' }}>
      <h2 style={{ color: '#0284C7' }}>< Streamed Layout Architectures
(Suspense)</h2>
      <p>This page streams component layout blocks over the connection
```

```

stream as they complete compilation.</p>

    { /* Renders instantly */
      <QuickSummaryPanel />

      { /* Renders a placeholder skeleton while the database executes, then
        swaps in the code dynamically */
        <Suspense fallback={<SkeletonLoader />}>
          <HeavyAnalyticsPanel />
        </Suspense>
      }
    }
  );
}

```

Step 7.3: Implement Global Route Streaming Options

Next.js also allows you to set up a global fallback template for the entire route layout by simply dropping a `loading.tsx` file in the directory. This catches the initial route load sequence.

Create a new file named `apps/frontend/src/app/analytics/loading.tsx` and add this code:

```

export default function GlobalRouteLoading() {
  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif', color:
      '#6B7280' }}>
      <h2>Loading App View Shell...</h2>
      <p>Next.js streaming connection established successfully.</p>
    </div>
  );
}

```

Step 7.4: Test the Live Progressive Streaming Layout

Launch the application development engine:

```
pnpm nx dev frontend
```

1. Open your browser and navigate to `http://localhost:4200/analytics`.
 2. Pay close attention to the screen layout sequence:
 - Instant: The global `loading.tsx` shell blinks briefly.
 - Fraction of a second later: The `QuickSummaryPanel` and the fallback `SkeletonLoader` render instantly onto your viewport.
 - Exactly 3.5 seconds later: The raw data chunk settles, and the loading dashboard morphs into the `HeavyAnalyticsPanel` smoothly—without causing a full-page browser refresh.
-